Project Design Phase-II
Technology Stack (Architecture & Stack)

Date	11 July 2026
Team ID	SWTID-2026-9508
Project Name	Credit Card Approval Prediction System
Maximum Marks	4 Marks

Technical Architecture:

The diagram below shows how a user's request flows through the Credit Card Approval Prediction System - from the browser, through the Flask application and preprocessing pipeline, into the trained ML model, and back as a prediction result.

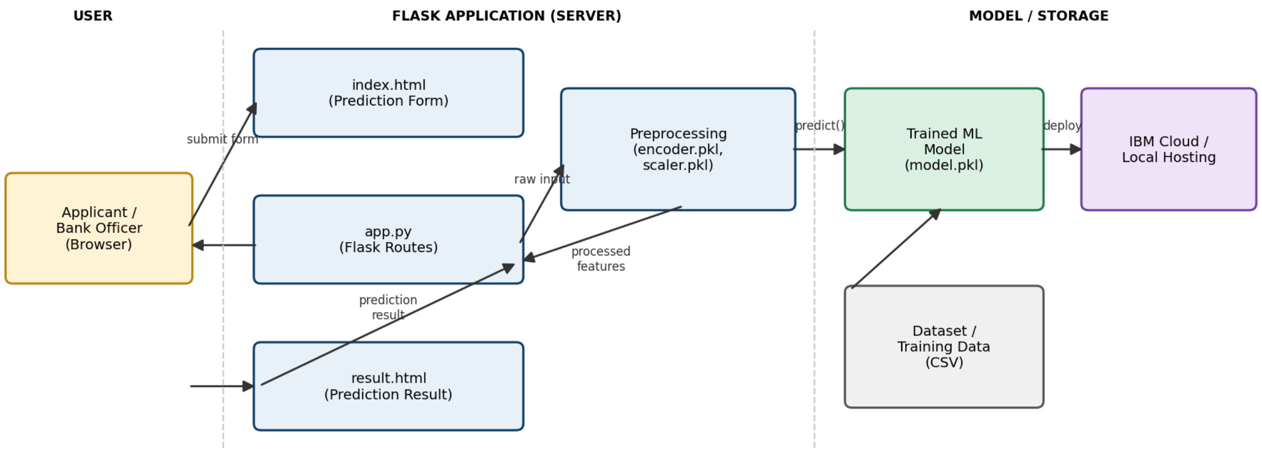

Table-1: Components & Technologies

S.No	Component	Description	Technology
1	User Interface	How the user interacts with the application	HTML, CSS, Bootstrap, Jinja2 templates (Flask)

S.No	Component	Description	Technology
2	Application Logic	Handles form submission, routes, and connects UI to the ML model	Python, Flask
3	Data Preprocessing	Encodes categorical fields and scales numerical fields before prediction	Scikit-learn (LabelEncoder / OneHotEncoder, StandardScaler)
4	Machine Learning Model	Predicts credit card Approval / Rejection from processed applicant data	Scikit-learn, XGBoost (best-performing model selected after comparison)
5	Model Persistence	Stores the trained model and preprocessing objects for reuse at inference time	Pickle / Joblib (model.pkl, encoder.pkl, scaler.pkl)
6	Dataset	Historical credit card application records used for training and evaluation	CSV file (Pandas)
7	File Storage	Storage of dataset, trained model files, and static assets	Local filesystem (project repository)
8	Infrastructure	Where the Flask application is deployed and hosted	Local server for development; IBM Cloud for deployment (if required)

Table-2: Application Characteristics

S.No	Characteristics	Description	Technology
1	Open-Source Frameworks	All core frameworks used are open-source	Flask, Scikit-learn, XGBoost, Pandas, NumPy, Bootstrap
2	Security	Basic input validation on the form; no sensitive applicant data is persisted beyond the prediction request	Flask input validation, WTForms (optional)
3	Scalable Architecture	Preprocessing and model inference are separated from the web layer, so the model can be retrained or swapped without changing the UI	Modular Flask app structure (routes / model / templates)
4	Availability	Lightweight Flask app with minimal dependencies keeps startup and response times low for demo and evaluation use	Flask development server / Gunicorn (for production)

S.No	Characteristics	Description	Technology
5	Performance	Model inference on a single applicant record is near-instant since the model is pre-trained and loaded once at app startup	In-memory model loading via Pickle/Joblib